

FINAL CAPSTONE

NAME – MANJUSHREE

DATE – 21-07-2026

Playwright UI Automation Project Report

Project title

End-to-End UI Automation Framework using Playwright with TypeScript

Objective

The objective of this project is to automate the complete user journey of an e-commerce web application using Playwright with TypeScript. The framework is designed in a modular and reusable way by separating locators, pages, flows, fixtures, test data, and logging functionality.

Technology stack

- **Automation Tool:** Playwright
 - **Programming Language:** TypeScript
 - **Framework:** Playwright Test
 - **IDE:** Visual Studio Code / IntelliJ IDEA
 - **Test Data Format:** JSON
 - **Reporting:** Playwright HTML Report
 - **Logging:** Custom Logger (logger.ts)
-

Modules used

Fixtures

Used to create reusable test setup and provide common objects across test cases.

Flows

Contains end-to-end business logic such as login, product search, add to cart, checkout, and payment.

Pages

Implements the Page Object Model (POM) design pattern where each page contains its actions and methods.

Locators

All UI locators are stored in separate files so they can be reused across pages and maintained easily.

Test data

Test data such as payment details are stored in JSON files to avoid hardcoding values in test scripts.

Logger

A centralized logger is used to maintain overall execution logs and improve debugging.

Test scenario automated

The following end-to-end scenario was automated:

1. Launch the application.
2. Log in using the demo user.
3. Search for products:
 - ProSound Wireless Headphones
 - ThinSlate 15 Laptop
 - QuickCharge 65W USB-C Hub
4. Add all products to the cart.
5. Increase and decrease product quantity in the cart.

6. Verify cart subtotal and total.
 7. Proceed to checkout.
 8. Enter payment details.
 9. Place the order.
 10. Verify order confirmation and order number.
 11. Continue shopping after successful order placement.
-

Logging

A custom logger (logger.ts) is used to print informational, warning, and error messages during test execution. This helps in tracking the execution flow and debugging failures.

Example log messages:

- [INFO] Logging in
 - [INFO] Searching for products
 - [INFO] Proceeding to checkout
 - [INFO] Payment completed successfully
-

Reporting

Playwright generates an HTML report after test execution. The report contains:

- Test case status (Pass/Fail)
- Execution time
- Error details
- Screenshots on failure (if configured)
- Trace information for debugging

Command to generate and open report

npx playwright show-report

Advantages of the framework

- Modular and reusable structure
 - Easy maintenance of locators
 - Separation of test logic and page actions
 - Centralized logging for debugging
 - Externalized test data using JSON
 - Scalable for adding new test cases
-

Conclusion

The Playwright UI Automation Framework successfully automates the complete e-commerce order placement workflow. The framework follows corporate-standard practices such as Page Object Model, centralized locators, reusable flows, fixtures, JSON-based test data, and centralized logging, making it easy to maintain and extend for future automation requirements.

FRAMEWORK STRUCTURE



